🚀 CompileX

Complete Source Code Documentation

Total Files

23

Total Lines of Code

3079

Total Characters

86,683

Generated

13/4/2026

📑 Table of Contents

- **backend/package.json** (16 lines)
- **backend/server.js** (91 lines)
- **backend/execute.js** (205 lines)
- **frontend/package.json** (31 lines)
- **frontend/vite.config.js** (8 lines)
- **frontend/eslint.config.js** (30 lines)
- **frontend/src\App.css** (256 lines)
- **frontend/src\App.jsx** (297 lines)
- **frontend/src\components\AIPanel\AIPanel.css** (395 lines)
- **frontend/src\components\AIPanel\AIPanel.jsx** (329 lines)
- **frontend/src\components\Console\Console.css** (268 lines)
- **frontend/src\components\Console\Console.jsx** (159 lines)
- **frontend/src\components\Editor\CodeEditor.css** (32 lines)
- **frontend/src\components\Editor\CodeEditor.jsx** (133 lines)
- **frontend/src\components\Navbar\Navbar.css** (226 lines)
- **frontend/src\components\Navbar\Navbar.jsx** (146 lines)
- **frontend/src\components\StatusBar\StatusBar.css** (71 lines)
- **frontend/src\components\StatusBar\StatusBar.jsx** (38 lines)
- **frontend/src\constants\languages.js** (132 lines)
- **frontend/src\index.css** (6 lines)
- **frontend/src\main.jsx** (11 lines)
- **frontend/src\services\gemini.js** (158 lines)
- **frontend/src\services\judge0.js** (41 lines)

📂 Source Files

📄 **backend/package.json** 16 lines

```
{
  "name": "compilex-backend",
  "version": "1.0.0",
  "description": "Execution Engine Backend",
  "main": "server.js",
  "type": "module",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.18.2",
    "socket.io": "^4.8.3"
  }
}
```

backend/server.js 91 lines

```
import express from 'express';
import cors from 'cors';
import http from 'http';
import { Server } from 'socket.io';
import { executeCodeLocally, executeInteractive } from './execute.js';

const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: '*' } });

const PORT = process.env.PORT || 5000;

app.use(cors());
app.use(express.json());

app.post('/execute', async (req, res) =&gt; {
    const { source_code, files, main_file, language_id, stdin } = req.body;

    // Support backwards compatibility
    const codeFiles = files || (source_code ? [{ name: 'main', content: source_code
}] : []);

    if (codeFiles.length === 0) {
        return res.status(400).json({ error: 'files or source_code is required' });
    }

    try {
        const result = await executeCodeLocally({ files: codeFiles, main_file,
language_id, stdin });
        res.json({
            stdout: result.stdout || '',
            stderr: result.stderr || '',
            compile_output: result.compile_output || '',
            time: result.time || '0.00',
            memory: result.memory || 0,
            status: { id: result.stderr || result.compile_output ? 11 : 3,
description: result.stderr ? 'Runtime Error' : 'Accepted' }
        });
    } catch (error) {
        console.error('Execution Error:', error);
        res.status(500).json({
            stdout: '',
            stderr: error.message || 'Internal Server Error',
            compile_output: '',
            time: '0.00',
            memory: 0,
            status: { id: 13, description: 'Internal Error' }
        });
    }
});
```

```
// WebSocket logic for interactive execution
io.on('connection', (socket) =&gt; {
    let activeProcess = null;

    socket.on('execute', async (data) =&gt; {
        if (activeProcess) {
            activeProcess.kill();
        }
        try {
            activeProcess = await executeInteractive(data, socket);
        } catch (error) {
            socket.emit('terminal_error', `Server Error: ${error.message}\n`);
            socket.emit('execution_complete', { status: 'error' });
        }
    });

    socket.on('input', (inputStr) =&gt; {
        if (activeProcess && activeProcess.stdin && !activeProcess.killed) {
            // Write to stdin
            activeProcess.stdin.write(inputStr + '\n');
        }
    });

    socket.on('stop', () =&gt; {
        if (activeProcess && !activeProcess.killed) {
            activeProcess.kill('SIGINT');
            socket.emit('terminal_error', '\n[Execution manually stopped]\n');
            socket.emit('execution_complete', { status: 'stopped' });
            activeProcess = null;
        }
    });

    socket.on('disconnect', () =&gt; {
        if (activeProcess && !activeProcess.killed) {
            activeProcess.kill('SIGKILL');
        }
    });
});

server.listen(PORT, () =&gt; {
    console.log(`Backend Execution Server running on http://localhost:${PORT}`);
});
```

backend/execute.js 205 lines

```
import fs from 'fs/promises';
import fsSync from 'fs';
import path from 'path';
import { exec, spawn } from 'child_process';
import util from 'util';
import os from 'os';

const execPromise = util.promisify(exec);

// Map Judge0 Language IDs to Extensions and Executors
const langMap = {
    71: { cmd: 'python', ext: 'py', runner: 'python' }, // Python
    63: { cmd: 'node', ext: 'js', runner: 'node' },     // Node.js
    62: { cmd: 'java', ext: 'java', runner: 'java' }    // Java
};

export const executeCodeLocally = async ({ files, main_file, language_id, stdin })
=&gt; {
    const lang = langMap[language_id];
    if (!lang) {
        throw new Error('Unsupported language ID.');
    }

    const tmpPath = path.join(os.tmpdir(),
`compilex_${Date.now()}_${Math.random().toString(36).substring(7)}`);
    await fs.mkdir(tmpPath, { recursive: true });

    let className = 'Main';
    let runFile = '';

    for (let i = 0; i &lt; files.length; i++) {
        const file = files[i];
        let fName = file.name;
        const sourceStr = Buffer.from(file.content, 'base64').toString('utf-8');

        // Retrofit legacy defaults
        if (fName === 'main' && lang.ext !== 'java') fName = 'main.' + lang.ext;

        if (lang.ext === 'java') {
            const match = sourceStr.match(/(?:public\s+)?class\s+([a-zA-Z0-9_]+)/);
            if (match) {
                const detectedName = match[1];
                // if this is the main file or only file, set as primary class
                if (file.name === main_file || files.length === 1 || fName ===
'main') {
                    className = detectedName;
                }
                if (!fName.endsWith('.java')) fName = detectedName + '.java';
            } else if (!fName.endsWith('.java')) {
                fName += '.java';
```

```
            }
        }

        if (file.name === main_file || (!runFile && i === 0)) {
            runFile = fName;
        }

        await fs.writeFile(path.join(tmpPath, fName), sourceStr);
    }

    // Always write input.txt so blocking read functions (like Scanner) hit EOF
instantly and crash
    // instead of waiting 10 seconds for user terminal input
    const stdinStr = stdin ? Buffer.from(stdin, 'base64').toString('utf-8') : '';
    await fs.writeFile(path.join(tmpPath, 'input.txt'), stdinStr);

    const startTime = process.hrtime();
    let result = { stdout: '', stderr: '', compile_output: '' };

    try {
        let command = '';
        const inputRedirect = ` &lt; input.txt`;

        // Java needs compilation first
        if (lang.ext === 'java') {
            // Compile
            try {
                // Ensure a lib directory exists for external JARs (though empty
initially)
                await fs.mkdir(path.join(tmpPath, 'lib'), { recursive: true });
                const compileCmd = os.platform() === 'win32'
                    ? `javac -cp ".;lib/*" *.java`
                    : `javac -cp ".:lib/*" *.java`;
                await execPromise(compileCmd, { cwd: tmpPath, timeout: 5000 });
            } catch (err) {
                result.compile_output = err.stderr || err.message;
                return calculateStats(result, startTime);
            }
            // Run
            const javaCp = os.platform() === 'win32' ? `".;lib/*"` : `".:lib/*"`;
            command = `java -cp ${javaCp} ${className}${inputRedirect}`;
        }
        else {
            command = `${lang.runner} ${runFile}${inputRedirect}`;
        }

        // Execute code
        const { stdout, stderr } = await execPromise(command, {
            cwd: tmpPath,
            timeout: 10000,
            maxBuffer: 1024 * 1024 * 5 // 5MB buffer
        });

        result.stdout = stdout;
```

```
        result.stderr = stderr;

        if (lang.ext === 'java' && stderr.includes('NoSuchElementException')) {
            result.stderr += '\n\n[CompileX Hint]: Java triggered a Scanner error.
Please provide your input in the Console "Input" tab before clicking Run.';
        }

    } catch (err) {
        if (err.killed) {
            result.stderr = 'Execution Time Limit Exceeded (10s)';
        } else {
            result.stderr = err.stderr || err.message;
        }
    } finally {
        // Cleanup temp files
        try {
            await fs.rm(tmpPath, { recursive: true, force: true });
        } catch (e) {
            console.error('Failed to cleanup temp dir:', e);
        }
    }

    return calculateStats(result, startTime);
};

function calculateStats(result, startTime) {
    const diff = process.hrtime(startTime);
    result.time = (diff[0] + diff[1] / 1e9).toFixed(3);
    result.memory = 32000; // Simulated memory limit
    return result;
}

export const executeInteractive = async (data, socket) =&gt; {
    const { files, main_file, language_id } = data;
    const lang = langMap[language_id];
    if (!lang) throw new Error('Unsupported language ID.');

    const tmpPath = path.join(os.tmpdir(),
`compilex_${Date.now()}_${Math.random().toString(36).substring(7)}`);
    await fs.mkdir(tmpPath, { recursive: true });

    let className = 'Main';
    let runFile = '';

    for (let i = 0; i &lt; files.length; i++) {
        const file = files[i];
        let fName = file.name;
        // Support base64 or unencoded payload seamlessly
        let sourceStr = file.content;
        try { sourceStr = Buffer.from(file.content, 'base64').toString('utf-8'); }
catch(e){}

        if (fName === 'main' && lang.ext !== 'java') fName = 'main.' + lang.ext;
```

```
        if (lang.ext === 'java') {
            const match = sourceStr.match(/(?:public\s+)?class\s+([a-zA-Z0-9_]+)/);
            if (match) {
                const detectedName = match[1];
                if (file.name === main_file || files.length === 1 || fName ===
'main') className = detectedName;
                if (!fName.endsWith('.java')) fName = detectedName + '.java';
            } else if (!fName.endsWith('.java')) {
                fName += '.java';
            }
        }
        if (file.name === main_file || (!runFile && i === 0)) runFile = fName;
        await fs.writeFile(path.join(tmpPath, fName), sourceStr);
    }

    const startTime = process.hrtime();
    let runnerCmd = lang.runner;
    let runnerArgs = [];

    if (lang.ext === 'java') {
        try {
            await fs.mkdir(path.join(tmpPath, 'lib'), { recursive: true });
            const compileCmd = os.platform() === 'win32' ? `javac -cp ".;lib/*"
*.java` : `javac -cp ".:lib/*" *.java`;
            await execPromise(compileCmd, { cwd: tmpPath });
        } catch (err) {
            socket.emit('terminal_error', err.stderr || err.message);
            socket.emit('execution_complete', calculateStats({ status: { id: 11 } },
startTime));
            return null;
        }
        const javaCp = os.platform() === 'win32' ? `.;lib/*` : `.:lib/*`;
        runnerCmd = 'java';
        runnerArgs = ['-cp', javaCp, className];
    } else if (lang.ext === 'py') {
        // Python caching disable -&gt; interactive stdout instantly
        runnerArgs = ['-u', runFile];
    } else {
        runnerArgs = [runFile];
    }

    const child = spawn(runnerCmd, runnerArgs, { cwd: tmpPath });

    child.stdout.on('data', (d) =&gt; socket.emit('terminal_output', d.toString()));
    child.stderr.on('data', (d) =&gt; socket.emit('terminal_error', d.toString()));
    child.on('error', (err) =&gt; socket.emit('terminal_error', err.message));

    child.on('close', (code) =&gt; {
        socket.emit('execution_complete', calculateStats({
            status: { id: code === 0 ? 3 : 11, description: code === 0 ? 'Accepted'
: 'Runtime Error' }
        }, startTime));
        // Cleanup
        fs.rm(tmpPath, { recursive: true, force: true }).catch(() =&gt; {});
```

```
    });

    return child;
};
```

frontend/package.json 31 lines

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "@monaco-editor/react": "^4.7.0",
    "axios": "^1.15.0",
    "react": "^19.2.4",
    "react-dom": "^19.2.4",
    "socket.io-client": "^4.8.3"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.4",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "eslint": "^9.39.4",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.4.0",
    "vite": "^8.0.4"
  }
}
```

frontend/vite.config.js 8 lines

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

frontend/eslint.config.js 30 lines

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
      parserOptions: {
        ecmaVersion: 'latest',
        ecmaFeatures: { jsx: true },
        sourceType: 'module',
      },
    },
    rules: {
      'no-unused-vars': ['error', { varsIgnorePattern: '^[A-Z_]' }],
    },
  },
])
```

```
/* ═══════════════════════════════════════════
   CompileX - Global Design System
   Dark glassmorphism IDE aesthetic
═══════════════════════════════════════════ */

@import url('https://fonts.googleapis.com/css2?
family=Inter:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;500;600&display
=swap');

/* ─── CSS Variables: Dark Theme ─── */
:root,
.dark {
  --accent: #7c3aed;
  --accent-light: #a78bfa;
  --accent-dim: rgba(124, 58, 237, 0.15);
  --teal: #06d6a0;
  --teal-dim: rgba(6, 214, 160, 0.15);

  --surface-0: #0a0a14;
  --surface-1: #0f0f1e;
  --surface-2: #161628;
  --surface-3: #1e1e35;
  --surface-glass: rgba(15, 15, 30, 0.85);

  --border: rgba(255, 255, 255, 0.07);
  --border-hover: rgba(255, 255, 255, 0.14);

  --text-primary: #e2e8f0;
  --text-secondary: #94a3b8;
  --text-muted: #4b5568;
}

/* ─── CSS Variables: Light Theme ─── */
.light {
  --accent: #7c3aed;
  --accent-light: #6d28d9;
  --accent-dim: rgba(124, 58, 237, 0.08);
  --teal: #059669;
  --teal-dim: rgba(5, 150, 105, 0.1);

  --surface-0: #ffffff;
  --surface-1: #f8fafc;
  --surface-2: #f1f5f9;
  --surface-3: #e2e8f0;
  --surface-glass: rgba(248, 250, 252, 0.9);

  --border: rgba(0, 0, 0, 0.08);
  --border-hover: rgba(0, 0, 0, 0.15);

  --text-primary: #0f172a;
```

```
  --text-secondary: #475569;
  --text-muted: #94a3b8;
}

/* ─── Reset & Base ─── */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

html, body, #root {
  height: 100%;
  width: 100%;
  overflow: hidden;
}

body {
  font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
  font-size: 14px;
  line-height: 1.5;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

/* ─── App Layout ─── */
.app {
  display: flex;
  flex-direction: column;
  height: 100vh;
  width: 100vw;
  background: var(--surface-0);
  color: var(--text-primary);
  overflow: hidden;
  transition: background 0.3s ease, color 0.3s ease;
  outline: none;
}

.app-body {
  display: flex;
  flex: 1;
  overflow: hidden;
  min-height: 0;
}

/* ─── Editor Area ─── */
.editor-area {
  display: flex;
  flex-direction: column;
  flex: 1;
  min-width: 0;
  overflow: hidden;
  position: relative;
}
```

```
/* ─── File Tabs ─── */
.file-tabs {
  display: flex;
  background: var(--surface-1);
  border-bottom: 1px solid var(--border);
  overflow-x: auto;
  flex-shrink: 0;
}
.file-tab {
  padding: 8px 16px;
  background: transparent;
  border: none;
  border-right: 1px solid var(--border);
  color: var(--text-muted);
  font-family: 'JetBrains Mono', monospace;
  font-size: 13px;
  cursor: pointer;
  display: flex;
  align-items: center;
  gap: 8px;
  transition: all 0.2s;
}
.file-tab.active {
  background: var(--surface-0);
  color: var(--text-primary);
  border-bottom: 2px solid var(--accent);
}
.file-tab:hover:not(.active) {
  background: var(--surface-2);
  color: var(--text-secondary);
}
.file-tab-close {
  background: transparent;
  border: none;
  color: inherit;
  cursor: pointer;
  border-radius: 4px;
  display: flex;
}
.file-tab-close:hover {
  background: rgba(239, 68, 68, 0.2);
  color: #ef4444;
}
.add-file-btn {
  padding: 8px 12px;
  background: transparent;
  border: none;
  color: var(--text-muted);
  cursor: pointer;
  transition: color 0.2s;
}
.add-file-btn:hover {
  color: var(--accent);
```

```
}

.editor-pane-container {
  display: flex;
  flex-direction: column;
  height: 100%;
}

.editor-pane {
  overflow: hidden;
  flex-shrink: 0;
  transition: height 0.01s;
}

.console-pane {
  overflow: hidden;
  flex-shrink: 0;
  transition: height 0.01s;
}

/* ─── Draggable Divider ─── */
.panel-divider {
  height: 5px;
  background: var(--border);
  cursor: row-resize;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
  transition: background 0.2s;
  position: relative;
  z-index: 10;
}

.panel-divider:hover {
  background: var(--accent-dim);
}

.panel-divider:hover .divider-handle span {
  background: var(--accent);
}

.divider-handle {
  display: flex;
  gap: 3px;
  align-items: center;
}

.divider-handle span {
  width: 22px;
  height: 2px;
  border-radius: 2px;
  background: var(--border-hover);
  transition: background 0.2s;
```

```
}

/* ─── Scrollbars ─── */
* {
  scrollbar-width: thin;
  scrollbar-color: var(--surface-3) transparent;
}

*::-webkit-scrollbar { width: 6px; height: 6px; }
*::-webkit-scrollbar-track { background: transparent; }
*::-webkit-scrollbar-thumb { background: var(--surface-3); border-radius: 3px; }
*::-webkit-scrollbar-thumb:hover { background: var(--accent-dim); }

/* ─── Animated Background Gradient (dark only) ─── */
.dark .app::before {
  content: '';
  position: fixed;
  top: -200px;
  right: -200px;
  width: 500px;
  height: 500px;
  background: radial-gradient(circle, rgba(124, 58, 237, 0.08) 0%, transparent 70%);
  pointer-events: none;
  z-index: 0;
  animation: float 8s ease-in-out infinite;
}

.dark .app::after {
  content: '';
  position: fixed;
  bottom: -150px;
  left: -100px;
  width: 400px;
  height: 400px;
  background: radial-gradient(circle, rgba(6, 214, 160, 0.06) 0%, transparent 70%);
  pointer-events: none;
  z-index: 0;
  animation: float 10s ease-in-out infinite reverse;
}

@keyframes float {
  0%, 100% { transform: translate(0, 0); }
  33% { transform: translate(20px, -20px); }
  66% { transform: translate(-15px, 15px); }
}
```

frontend/src\App.jsx 297 lines

```
import React, { useState, useCallback, useRef, useEffect } from 'react';
import Navbar from './components/Navbar/Navbar';
import CodeEditor from './components/Editor/CodeEditor';
import Console from './components/Console/Console';
import AIPanel from './components/AIPanel/AIPanel';
import StatusBar from './components/StatusBar/StatusBar';
import { LANGUAGES, getLanguageById } from './constants/languages';
import { io } from 'socket.io-client';
import './App.css';

const socket = io('http://localhost:5000', { autoConnect: false });

// Initialize files grouped by language to avoid template overwrite bug
const initialFiles = LANGUAGES.reduce((acc, lang) =&gt; {
  const ext = lang.id === 'python' ? '.py' : lang.id === 'javascript' ? '.js' :
lang.id === 'cpp' ? '.cpp' : lang.id === 'c' ? '.c' : '.java';
  const mainName = lang.id === 'java' ? 'Main.java' : 'main' + ext;
  acc[lang.id] = [{ name: mainName, content: lang.defaultCode }];
  return acc;
}, {});

export default function App() {
  const [theme, setTheme] = useState('dark');
  const [selectedLanguage, setSelectedLanguage] = useState('python');

  const [filesByLang, setFilesByLang] = useState(initialFiles);
  const [activeFileName, setActiveFileName] = useState('main.py');
  const [terminalData, setTerminalData] = useState([]);
  const [result, setResult] = useState(null);
  const [isRunning, setIsRunning] = useState(false);
  const [isInteractive, setIsInteractive] = useState(false);
  const [aiPanelOpen, setAIPanelOpen] = useState(false);
  const [consoleSize, setConsoleSize] = useState(35); // % of vertical split
  const isDragging = useRef(false);
  const containerRef = useRef(null);

  // Derived active file properties
  const activeLangFiles = filesByLang[selectedLanguage] || [];
  const activeFile = activeLangFiles.find(f =&gt; f.name === activeFileName) ||
activeLangFiles[0];
  const code = activeFile ? activeFile.content : '';

  // Synchronize active file on language change
  useEffect(() =&gt; {
    const defaultFiles = filesByLang[selectedLanguage];
    if (defaultFiles && defaultFiles.length &gt; 0) {
      setActiveFileName(defaultFiles[0].name);
    }
  }, [selectedLanguage]);
```

```
const setCode = (newContent) =&gt; {
  setFilesByLang(prev =&gt; {
    const langFiles = [...prev[selectedLanguage]];
    const idx = langFiles.findIndex(f =&gt; f.name === activeFile.name);
    if (idx !== -1) {
      langFiles[idx] = { ...langFiles[idx], content: newContent };
    }
    return { ...prev, [selectedLanguage]: langFiles };
  });
};

const handleAddFile = () =&gt; {
  const name = prompt('Enter new file name:');
  if (!name?.trim()) return;
  if (activeLangFiles.some(f =&gt; f.name === name)) {
    alert('File already exists.');
    return;
  }
  setFilesByLang(prev =&gt; ({
    ...prev,
    [selectedLanguage]: [...prev[selectedLanguage], { name, content: '' }]
  }));
  setActiveFileName(name);
};

const handleDeleteFile = (e, name) =&gt; {
  e.stopPropagation();
  if (activeLangFiles.length === 1) return;
  setFilesByLang(prev =&gt; {
    const newFiles = prev[selectedLanguage].filter(f =&gt; f.name !== name);
    return { ...prev, [selectedLanguage]: newFiles };
  });
  if (activeFileName === name) setActiveFileName(activeLangFiles[0].name);
};

// Setup WebSockets
useEffect(() =&gt; {
  socket.connect();

  socket.on('terminal_output', data =&gt; {
    setTerminalData(prev =&gt; [...prev, { type: 'output', text: data }]);
  });

  socket.on('terminal_error', data =&gt; {
    setTerminalData(prev =&gt; [...prev, { type: 'error', text: data }]);
  });

  socket.on('execution_complete', data =&gt; {
    setIsRunning(false);
    setIsInteractive(false);
    setResult(data);
  });

  return () =&gt; {
```

```
    socket.off('terminal_output');
    socket.off('terminal_error');
    socket.off('execution_complete');
  };
}, []);

// Language switch
const handleLanguageChange = (langId) =&gt; {
  setSelectedLanguage(langId);
  setResult(null);
  setTerminalData([]);
};

// Run code via WebSocket
const handleRun = useCallback(() =&gt; {
  if (isRunning) return;
  setIsRunning(true);
  setIsInteractive(true);
  setResult(null);
  setTerminalData([]); // clear previous output

  const encode = (str) =&gt; btoa(unescape(encodeURIComponent(str)));
  const encodedFiles = activeLangFiles.map(f =&gt; ({
    name: f.name,
      content: encode(f.content)
  }));

  const lang = getLanguageById(selectedLanguage);
  socket.emit('execute', {
      files: encodedFiles,
      main_file: activeFileName,
      language_id: lang.judge0Id
  });
}, [activeLangFiles, activeFileName, selectedLanguage, isRunning]);

const handleStop = useCallback(() =&gt; {
  socket.emit('stop');
}, []);

const handleTerminalInput = useCallback((text) =&gt; {
  socket.emit('input', text);
  setTerminalData(prev =&gt; [...prev, { type: 'input', text: text + '\n' }]);
}, []);

// AI insert code into editor
const handleInsertCode = useCallback((insertedCode) =&gt; {
  setCode(insertedCode);
}, []);

// Keyboard shortcut: Ctrl+Enter to run
const handleKeyDown = useCallback(
  (e) =&gt; {
    if ((e.ctrlKey || e.metaKey) && e.key === 'Enter') {
      e.preventDefault();
```

```
        handleRun();
      }
    },
    [handleRun]
  );

  // Draggable divider for resizing panels
  const handleDividerMouseDown = (e) =&gt; {
    e.preventDefault();
    isDragging.current = true;
    document.body.style.cursor = 'row-resize';
    document.body.style.userSelect = 'none';

    const onMouseMove = (ev) =&gt; {
      if (!isDragging.current || !containerRef.current) return;
      const rect = containerRef.current.getBoundingClientRect();
      const ratio = ((ev.clientY - rect.top) / rect.height) * 100;
      const clamped = Math.min(Math.max(ratio, 20), 80);
      setConsoleSize(100 - clamped);
    };

    const onMouseUp = () =&gt; {
      isDragging.current = false;
      document.body.style.cursor = '';
      document.body.style.userSelect = '';
      window.removeEventListener('mousemove', onMouseMove);
      window.removeEventListener('mouseup', onMouseUp);
    };

    window.addEventListener('mousemove', onMouseMove);
    window.addEventListener('mouseup', onMouseUp);
  };

  const execError = result ? (result.stderr || result.compile_output || '') : '';
  const lineCount = code.split('\n').length;

  return (
    &lt;div
      className={`app ${theme}`}
      onKeyDown={handleKeyDown}
      tabIndex={-1}
    &gt;
      {/* Navbar */}
      &lt;Navbar
        selectedLanguage={selectedLanguage}
        onLanguageChange={handleLanguageChange}
        onRun={handleRun}
        isRunning={isRunning}
        theme={theme}
        onThemeToggle={() =&gt; setTheme((t) =&gt; (t === 'dark' ? 'light' :
'dark'))}
        onAIToggle={() =&gt; setAIPanelOpen((o) =&gt; !o)}
        aiPanelOpen={aiPanelOpen}
      /&gt;
```

```
      {/* Main layout */}
      &lt;div className="app-body"&gt;
        {/* Editor + Console column */}
        &lt;div className="editor-area" ref={containerRef}&gt;
          {/* File Tabs UI & Monaco Editor */}
          &lt;div className="editor-pane-container" style={{ height: `${100 -
consoleSize}%` }}&gt;
            &lt;div className="file-tabs"&gt;
              {activeLangFiles.map(f =&gt; (
                &lt;button
                  key={f.name}
                  className={`file-tab ${activeFileName === f.name ? 'active' :
''}`}
                  onClick={() =&gt; setActiveFileName(f.name)}
                &gt;
                  {f.name}
                  {activeLangFiles.length &gt; 1 && (
                    &lt;span
                      className="file-tab-close"
                      title="Delete file"
                      onClick={(e) =&gt; handleDeleteFile(e, f.name)}
                    &gt;
                      &lt;svg width="10" height="10" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2"&gt;
                        &lt;line x1="18" y1="6" x2="6" y2="18" /&gt;&lt;line x1="6"
y1="6" x2="18" y2="18" /&gt;
                      &lt;/svg&gt;
                    &lt;/span&gt;
                  )}
                &lt;/button&gt;
              ))}
              &lt;button className="add-file-btn" onClick={handleAddFile} title="Add
new file"&gt;
                &lt;svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2.5"&gt;
                  &lt;line x1="12" y1="5" x2="12" y2="19" /&gt;&lt;line x1="5"
y1="12" x2="19" y2="12" /&gt;
                &lt;/svg&gt;
              &lt;/button&gt;
            &lt;/div&gt;
            &lt;div className="editor-pane" style={{ flex: 1 }}&gt;
              &lt;CodeEditor
                value={code}
                language={getLanguageById(selectedLanguage).monacoId}
                theme={theme}
                onChange={(val) =&gt; setCode(val || '')}
              /&gt;
            &lt;/div&gt;
          &lt;/div&gt;

          {/* Draggable Divider */}
          &lt;div className="panel-divider" onMouseDown={handleDividerMouseDown}&gt;
            &lt;div className="divider-handle"&gt;
```

```
              &lt;span /&gt;&lt;span /&gt;&lt;span /&gt;
            &lt;/div&gt;
          &lt;/div&gt;

          {/* Output Console */}
          &lt;div className="console-pane" style={{ height: `${consoleSize}%` }}&gt;
            &lt;Console
              terminalData={terminalData}
              result={result}
              isRunning={isRunning}
              isInteractive={isInteractive}
              onSendInput={handleTerminalInput}
              onStop={handleStop}
            /&gt;
          &lt;/div&gt;
        &lt;/div&gt;

        {/* AI Panel */}
        {aiPanelOpen && (
          &lt;AIPanel
            isOpen={aiPanelOpen}
            code={code}
            files={activeLangFiles}
            language={selectedLanguage}
            execError={execError}
            onInsertCode={handleInsertCode}
            onClose={() =&gt; setAIPanelOpen(false)}
          /&gt;
        )}
      &lt;/div&gt;

      {/* Status Bar */}
      &lt;StatusBar
        selectedLanguage={selectedLanguage}
        linesCount={lineCount}
        charCount={code.length}
      /&gt;
    &lt;/div&gt;
  );
}
```

frontend/src\components\AIPanel\AIPanel.css 395 lines

```
.ai-panel {
  display: flex;
  flex-direction: column;
  width: 360px;
  min-width: 320px;
  height: 100%;
  background: var(--surface-1);
  border-left: 1px solid var(--border);
  overflow: hidden;
  animation: slideIn 0.25s cubic-bezier(0.16, 1, 0.3, 1);
  flex-shrink: 0;
}

@keyframes slideIn {
  from { transform: translateX(100%); opacity: 0; }
  to { transform: translateX(0); opacity: 1; }
}

/* Header */
.ai-panel-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.875rem 1rem;
  border-bottom: 1px solid var(--border);
  background: var(--surface-glass);
  backdrop-filter: blur(10px);
  flex-shrink: 0;
}

.ai-panel-title {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}

.ai-avatar {
  width: 30px;
  height: 30px;
  border-radius: 8px;
  background: linear-gradient(135deg, var(--accent), #a855f7);
  display: flex;
  align-items: center;
  justify-content: center;
  color: white;
  flex-shrink: 0;
  box-shadow: 0 0 12px rgba(124, 58, 237, 0.4);
}

.ai-panel-name {
```

```
  display: block;
  font-size: 0.875rem;
  font-weight: 600;
  color: var(--text-primary);
  line-height: 1.2;
}

.ai-panel-model {
  display: block;
  font-size: 0.7rem;
  color: var(--text-muted);
  line-height: 1.2;
}

.ai-close-btn {
  width: 26px;
  height: 26px;
  display: flex;
  align-items: center;
  justify-content: center;
  background: var(--surface-2);
  border: 1px solid var(--border);
  border-radius: 6px;
  color: var(--text-muted);
  cursor: pointer;
  transition: all 0.15s;
}

.ai-close-btn:hover {
  background: var(--surface-3);
  color: var(--text-primary);
}

/* Quick Actions */
.quick-actions {
  display: flex;
  gap: 0.375rem;
  padding: 0.75rem;
  overflow-x: auto;
  flex-shrink: 0;
  scrollbar-width: none;
  border-bottom: 1px solid var(--border);
}

.quick-actions::-webkit-scrollbar { display: none; }

.quick-action-btn {
  display: flex;
  align-items: center;
  gap: 0.375rem;
  white-space: nowrap;
  padding: 0.375rem 0.75rem;
  border-radius: 20px;
  border: 1px solid var(--border);
```

```
  background: var(--surface-2);
  color: var(--text-secondary);
  font-size: 0.78rem;
  font-weight: 500;
  font-family: inherit;
  cursor: pointer;
  transition: all 0.2s ease;
  flex-shrink: 0;
}

.quick-action-btn:hover {
  border-color: var(--accent);
  color: var(--accent-light);
  background: rgba(124, 58, 237, 0.1);
}

.quick-action-btn.active {
  border-color: var(--accent);
  background: rgba(124, 58, 237, 0.15);
  color: var(--accent-light);
  box-shadow: 0 0 10px rgba(124, 58, 237, 0.2);
}

.quick-action-btn:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

/* Messages */
.ai-messages {
  flex: 1;
  overflow-y: auto;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 1rem;
  scrollbar-width: thin;
  scrollbar-color: var(--border) transparent;
}

.ai-message {
  display: flex;
  align-items: flex-start;
  gap: 0.625rem;
}

.ai-message.user {
  flex-direction: row-reverse;
}

.ai-avatar-small {
  width: 24px;
  height: 24px;
  border-radius: 6px;
```

```
  background: linear-gradient(135deg, var(--accent), #a855f7);
  display: flex;
  align-items: center;
  justify-content: center;
  color: white;
  font-size: 0.7rem;
  flex-shrink: 0;
  margin-top: 2px;
}

.message-bubble {
  flex: 1;
  max-width: 100%;
  min-width: 0;
}

.ai-message.user .message-bubble {
  background: linear-gradient(135deg, var(--accent), #a855f7);
  border-radius: 12px 4px 12px 12px;
  padding: 0.625rem 0.875rem;
  color: white;
  font-size: 0.825rem;
  line-height: 1.5;
  max-width: 85%;
  word-break: break-word;
}

/* Message Content */
.message-content {
  font-size: 0.825rem;
  line-height: 1.65;
  color: var(--text-secondary);
}

.message-content p { margin: 0 0 0.5rem 0; }
.message-content p:last-child { margin-bottom: 0; }
.message-content h3 { font-size: 0.9rem; color: var(--text-primary); margin: 0.75rem
0 0.375rem; }
.message-content h4 { font-size: 0.825rem; color: var(--text-primary); margin:
0.5rem 0 0.25rem; }
.message-content li { margin: 0.2rem 0 0.2rem 1rem; color: var(--text-secondary); }
.message-content blockquote { border-left: 3px solid var(--accent); padding-left:
0.75rem; color: var(--text-muted); font-style: italic; margin: 0.5rem 0; }
.message-content strong { color: var(--text-primary); }

.text-block { margin-bottom: 0.25rem; }

/* Code blocks */
.code-block-wrapper {
  background: var(--surface-0);
  border: 1px solid var(--border);
  border-radius: 8px;
  overflow: hidden;
  margin: 0.5rem 0;
```

```
}

.code-block-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.375rem 0.75rem;
  background: var(--surface-2);
  border-bottom: 1px solid var(--border);
}

.code-lang {
  font-size: 0.7rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--accent-light);
}

.insert-btn {
  display: flex;
  align-items: center;
  gap: 0.25rem;
  font-size: 0.7rem;
  font-family: inherit;
  font-weight: 500;
  color: var(--teal);
  background: rgba(6, 214, 160, 0.1);
  border: 1px solid rgba(6, 214, 160, 0.3);
  border-radius: 4px;
  padding: 2px 7px;
  cursor: pointer;
  transition: all 0.15s;
}

.insert-btn:hover {
  background: rgba(6, 214, 160, 0.2);
  border-color: var(--teal);
}

.code-block {
  margin: 0;
  padding: 0.75rem;
  font-family: 'JetBrains Mono', 'Fira Code', monospace;
  font-size: 0.775rem;
  line-height: 1.5;
  color: var(--text-primary);
  overflow-x: auto;
  white-space: pre-wrap;
  word-break: break-word;
  max-height: 250px;
  overflow-y: auto;
}
```

```
/* Typing Indicator */
.ai-typing {
  display: flex;
  gap: 4px;
  align-items: center;
  padding: 0.5rem 0;
}

.ai-typing span {
  width: 7px;
  height: 7px;
  background: var(--accent);
  border-radius: 50%;
  animation: bounce 1.2s ease infinite;
}

.ai-typing span:nth-child(2) { animation-delay: 0.2s; }
.ai-typing span:nth-child(3) { animation-delay: 0.4s; }

@keyframes bounce {
  0%, 60%, 100% { transform: translateY(0); opacity: 0.4; }
  30% { transform: translateY(-6px); opacity: 1; }
}

/* Input Area */
.ai-input-area {
  padding: 0.75rem;
  border-top: 1px solid var(--border);
  background: var(--surface-glass);
  backdrop-filter: blur(10px);
  flex-shrink: 0;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.ai-prompt-form {
  display: flex;
  gap: 0.5rem;
  align-items: flex-end;
}

.ai-prompt-input {
  flex: 1;
  background: var(--surface-2);
  border: 1px solid var(--border);
  border-radius: 10px;
  color: var(--text-primary);
  font-family: inherit;
  font-size: 0.825rem;
  line-height: 1.5;
  padding: 0.625rem 0.875rem;
  resize: none;
  outline: none;
```

```
  transition: border-color 0.2s, box-shadow 0.2s;
  min-height: 56px;
  max-height: 120px;
  overflow-y: auto;
}

.ai-prompt-input:focus {
  border-color: var(--accent);
  box-shadow: 0 0 0 3px rgba(124, 58, 237, 0.1);
}

.ai-prompt-input::placeholder { color: var(--text-muted); }

.ai-submit-btn {
  width: 36px;
  height: 36px;
  border-radius: 10px;
  background: linear-gradient(135deg, var(--accent), #a855f7);
  border: none;
  color: white;
  display: flex;
  align-items: center;
  justify-content: center;
  cursor: pointer;
  flex-shrink: 0;
  transition: all 0.2s;
  box-shadow: 0 2px 8px rgba(124, 58, 237, 0.3);
}

.ai-submit-btn:hover { transform: scale(1.05); box-shadow: 0 4px 12px rgba(124, 58,
237, 0.4); }
.ai-submit-btn:disabled { opacity: 0.5; cursor: not-allowed; transform: none; }

/* Convert Selector */
.convert-selector {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  background: var(--surface-2);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 0.5rem 0.75rem;
}

.convert-selector label {
  font-size: 0.78rem;
  color: var(--text-muted);
  white-space: nowrap;
}

.convert-select {
  flex: 1;
  background: transparent;
  border: none;
```

```
  color: var(--text-primary);
  font-size: 0.825rem;
  font-family: inherit;
  outline: none;
  cursor: pointer;
}

.convert-select option { background: var(--surface-1); }

.convert-go-btn {
  padding: 0.25rem 0.75rem;
  background: linear-gradient(135deg, var(--accent), #a855f7);
  border: none;
  border-radius: 6px;
  color: white;
  font-size: 0.78rem;
  font-weight: 600;
  font-family: inherit;
  cursor: pointer;
  transition: all 0.2s;
}

.convert-go-btn:hover { opacity: 0.9; }
.convert-go-btn:disabled { opacity: 0.5; cursor: not-allowed; }
```

📄 **frontend/src\components\AIPanel\AIPanel.jsx** 329 lines

```
import React, { useState, useRef, useEffect } from 'react';
import { generateCode, debugCode, explainCode, optimizeCode, convertCode,
isAIMockMode } from '../../services/gemini';
import { LANGUAGES } from '../../constants/languages';
import './AIPanel.css';

const quickActions = [
  { id: 'generate', label: 'Generate Code', icon: '✨', desc: 'Describe what to
build' },
  { id: 'debug', label: 'Debug', icon: '🐛', desc: 'Find and fix bugs' },
  { id: 'explain', label: 'Explain', icon: '📖', desc: 'Understand the code' },
  { id: 'optimize', label: 'Optimize', icon: '⚡', desc: 'Improve performance' },
  { id: 'convert', label: 'Convert', icon: '🔄', desc: 'Translate to another
language' },
];

// Simple markdown-to-text renderer for code blocks
function MessageContent({ text, onInsert }) {
  if (!text) return null;

  // Split on code blocks
  const parts = text.split(/(```[\w]*\n[\s\S]*?```)/g);

  return (
    &lt;div className="message-content"&gt;
      {parts.map((part, i) =&gt; {
        const codeMatch = part.match(/^```([\w]*)\n([\s\S]*?)```$/);
        if (codeMatch) {
          const lang = codeMatch[1] || '';
          const code = codeMatch[2].trimEnd();
          return (
            &lt;div key={i} className="code-block-wrapper"&gt;
              &lt;div className="code-block-header"&gt;
                &lt;span className="code-lang"&gt;{lang || 'code'}&lt;/span&gt;
                &lt;button className="insert-btn" onClick={() =&gt; onInsert(code)}
title="Insert into editor"&gt;
                  &lt;svg width="12" height="12" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2"&gt;
                    &lt;path d="M12 5v14M5 12l7 7 7-7" /&gt;
                  &lt;/svg&gt;
                  Insert
                &lt;/button&gt;
              &lt;/div&gt;
              &lt;pre className="code-block"&gt;&lt;code&gt;
{code}&lt;/code&gt;&lt;/pre&gt;
            &lt;/div&gt;
          );
        }
        // Render text with basic markdown (bold, headers)
        return (
```

```
          &lt;div key={i} className="text-block"&gt;
            {part.split('\n').map((line, j) =&gt; {
              if (line.startsWith('## ')) return &lt;h3 key={j}&gt;
{line.slice(3)}&lt;/h3&gt;;
              if (line.startsWith('### ')) return &lt;h4 key={j}&gt;
{line.slice(4)}&lt;/h4&gt;;
              if (line.startsWith('&gt; ')) return &lt;blockquote key={j}&gt;
{line.slice(2)}&lt;/blockquote&gt;;
              if (line.startsWith('- ') || line.startsWith('* ')) return &lt;li key=
{j}&gt;{parseBold(line.slice(2))}&lt;/li&gt;;
              if (line.match(/^\d+\. /)) return &lt;li key={j}&gt;
{parseBold(line.replace(/^\d+\. /, ''))}&lt;/li&gt;;
              if (line.startsWith('**') && line.endsWith('**')) return &lt;p key=
{j}&gt;&lt;strong&gt;{line.slice(2, -2)}&lt;/strong&gt;&lt;/p&gt;;
              return line ? &lt;p key={j}&gt;{parseBold(line)}&lt;/p&gt; : &lt;br
key={j} /&gt;;
            })}
          &lt;/div&gt;
        );
      })}
    &lt;/div&gt;
  );
}

function parseBold(text) {
  const parts = text.split(/(\*\*[\s\S]*?\*\*)/g);
  return parts.map((p, i) =&gt;
    p.startsWith('**') && p.endsWith('**')
      ? &lt;strong key={i}&gt;{p.slice(2, -2)}&lt;/strong&gt;
      : p
  );
}

export default function AIPanel({ isOpen, code, files, language, execError,
onInsertCode, onClose }) {
  const [messages, setMessages] = useState([
    {
      role: 'ai',
      text: `👋 Hi! I'm your **CompileX AI Assistant** powered by Gemini.\n\nI can
help you:\n- ✨ **Generate** code from your description\n- 🐛 **Debug** errors and
fix bugs\n- 📖 **Explain** code line by line\n- ⚡ **Optimize** for better
performance\n- 🔄 **Convert** code between languages\n\nWhat would you like to do?
${isAIMockMode() ? '\n\n&gt; ⚠️ **Demo Mode** — Add `VITE_GEMINI_API_KEY` to `.env`
for real AI responses.' : ''}`,
    },
  ]);
  const [activeAction, setActiveAction] = useState(null);
  const [prompt, setPrompt] = useState('');
  const [convertTarget, setConvertTarget] = useState('javascript');
  const [isLoading, setIsLoading] = useState(false);
  const bottomRef = useRef(null);
  const inputRef = useRef(null);

  useEffect(() =&gt; {
```

```
    bottomRef.current?.scrollIntoView({ behavior: 'smooth' });
  }, [messages]);

  useEffect(() =&gt; {
    if (activeAction && inputRef.current) inputRef.current.focus();
  }, [activeAction]);

  const currentLang = LANGUAGES.find((l) =&gt; l.id === language) || LANGUAGES[0];

  const addMessage = (role, text) =&gt; {
    setMessages((prev) =&gt; [...prev, { role, text }]);
  };

  const handleAction = async (actionId) =&gt; {
    setActiveAction(actionId);
    if (actionId === 'generate') {
      setPrompt('');
    } else {
      // Auto-trigger for non-prompt actions
      await runAction(actionId, null);
    }
  };

  const runAction = async (actionId, userPrompt) =&gt; {
    setIsLoading(true);
    const action = actionId || activeAction;
    const promptText = userPrompt !== null ? userPrompt : prompt;

    let userMsg = '';
    let aiResult = null;

    try {
      // Use files array if available, otherwise fallback to backward compatibility
with single code block
      const contextBundle = files && files.length &gt; 0 ? files : code;

      switch (action) {
        case 'generate':
          userMsg = `Generate ${currentLang.label} code: ${promptText}`;
          aiResult = await generateCode(promptText, currentLang.label);
          break;
        case 'debug':
          userMsg = `Debug my ${currentLang.label} code`;
          aiResult = await debugCode(contextBundle, execError || '',
currentLang.label);
          break;
        case 'explain':
          userMsg = `Explain my ${currentLang.label} code`;
          aiResult = await explainCode(contextBundle, currentLang.label);
          break;
        case 'optimize':
          userMsg = `Optimize my ${currentLang.label} code`;
          aiResult = await optimizeCode(contextBundle, currentLang.label);
          break;
```

```
        case 'convert':
          const targetLang = LANGUAGES.find((l) =&gt; l.id === convertTarget);
          userMsg = `Convert my ${currentLang.label} code to ${targetLang?.label}`;
          aiResult = await convertCode(contextBundle, currentLang.label,
targetLang?.label || 'JavaScript');
          break;
        default:
          aiResult = { text: 'Unknown action.', type: 'text' };
      }

      if (userMsg) addMessage('user', userMsg);
      addMessage('ai', aiResult.text);
    } catch (err) {
      addMessage('ai', `❌ **Error**: ${err.message}\n\nPlease check your API key
configuration.`);
    } finally {
      setIsLoading(false);
      setActiveAction(null);
      setPrompt('');
    }
  };

  const handleSubmit = (e) =&gt; {
    e.preventDefault();
    if (!prompt.trim()) return;
    runAction(activeAction, prompt);
  };

  if (!isOpen) return null;

  return (
    &lt;aside className="ai-panel"&gt;
      {/* Header */}
      &lt;div className="ai-panel-header"&gt;
        &lt;div className="ai-panel-title"&gt;
          &lt;div className="ai-avatar"&gt;
            &lt;svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2"&gt;
              &lt;path d="M12 2a10 10 0 1 0 10 10A10 10 0 0 0 12 2z" /&gt;
              &lt;path d="M8 14s1.5 2 4 2 4-2 4-2" /&gt;
              &lt;line x1="9" y1="9" x2="9.01" y2="9" strokeWidth="3"
strokeLinecap="round" /&gt;
              &lt;line x1="15" y1="9" x2="15.01" y2="9" strokeWidth="3"
strokeLinecap="round" /&gt;
            &lt;/svg&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;span className="ai-panel-name"&gt;AI Assistant&lt;/span&gt;
            &lt;span className="ai-panel-model"&gt;{isAIMockMode() ? 'Demo Mode' :
'Gemini 1.5 Flash'}&lt;/span&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        &lt;button className="ai-close-btn" onClick={onClose} id="ai-panel-
close"&gt;
```

```
          &lt;svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2"&gt;
            &lt;line x1="18" y1="6" x2="6" y2="18" /&gt;&lt;line x1="6" y1="6"
x2="18" y2="18" /&gt;
          &lt;/svg&gt;
        &lt;/button&gt;
      &lt;/div&gt;

      {/* Quick Actions */}
      &lt;div className="quick-actions"&gt;
        {quickActions.map((action) =&gt; (
          &lt;button
            key={action.id}
            id={`ai-action-${action.id}`}
            className={`quick-action-btn ${activeAction === action.id ? 'active' :
''}`}
            onClick={() =&gt; handleAction(action.id)}
            disabled={isLoading}
            title={action.desc}
          &gt;
            &lt;span&gt;{action.icon}&lt;/span&gt;
            &lt;span&gt;{action.label}&lt;/span&gt;
          &lt;/button&gt;
        ))}
      &lt;/div&gt;

      {/* Messages */}
      &lt;div className="ai-messages"&gt;
        {messages.map((msg, i) =&gt; (
          &lt;div key={i} className={`ai-message ${msg.role}`}&gt;
            {msg.role === 'ai' && (
              &lt;div className="message-avatar ai-avatar-small"&gt;
                &lt;span&gt;✦&lt;/span&gt;
              &lt;/div&gt;
            )}
            &lt;div className="message-bubble"&gt;
              &lt;MessageContent text={msg.text} onInsert={onInsertCode} /&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        ))}
        {isLoading && (
          &lt;div className="ai-message ai"&gt;
            &lt;div className="message-avatar ai-avatar-
small"&gt;&lt;span&gt;✦&lt;/span&gt;&lt;/div&gt;
            &lt;div className="message-bubble"&gt;
              &lt;div className="ai-typing"&gt;
                &lt;span /&gt;&lt;span /&gt;&lt;span /&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        )}
        &lt;div ref={bottomRef} /&gt;
      &lt;/div&gt;
```

```
      {/* Input Area */}
      &lt;div className="ai-input-area"&gt;
        {activeAction === 'convert' && (
          &lt;div className="convert-selector"&gt;
            &lt;label&gt;Convert to:&lt;/label&gt;
            &lt;select
              id="convert-target-lang"
              value={convertTarget}
              onChange={(e) =&gt; setConvertTarget(e.target.value)}
              className="convert-select"
            &gt;
              {LANGUAGES.filter((l) =&gt; l.id !== language).map((l) =&gt; (
                &lt;option key={l.id} value={l.id}&gt;{l.icon}
{l.label}&lt;/option&gt;
              ))}
            &lt;/select&gt;
            &lt;button
              className="convert-go-btn"
              onClick={() =&gt; runAction('convert', null)}
              disabled={isLoading}
            &gt;
              Convert
            &lt;/button&gt;
          &lt;/div&gt;
        )}

        {activeAction === 'generate' && (
          &lt;form className="ai-prompt-form" onSubmit={handleSubmit}&gt;
            &lt;textarea
              ref={inputRef}
              id="ai-prompt-input"
              className="ai-prompt-input"
              value={prompt}
              onChange={(e) =&gt; setPrompt(e.target.value)}
              placeholder={`Describe what to generate in ${currentLang.label}…`}
              onKeyDown={(e) =&gt; {
                if (e.key === 'Enter' && !e.shiftKey) {
                  e.preventDefault();
                  handleSubmit(e);
                }
              }}
              disabled={isLoading}
              rows={2}
            /&gt;
            &lt;button
              id="ai-submit-btn"
              type="submit"
              className="ai-submit-btn"
              disabled={isLoading || !prompt.trim()}
            &gt;
              &lt;svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor"&gt;
                &lt;path d="M2 21l21-9L2 3v7l15 2-15 2z" /&gt;
              &lt;/svg&gt;
```

```
            &lt;/button&gt;
          &lt;/form&gt;
        )}

        {!activeAction && (
          &lt;form className="ai-prompt-form" onSubmit={handleSubmit}&gt;
            &lt;textarea
              ref={inputRef}
              id="ai-chat-input"
              className="ai-prompt-input"
              value={prompt}
              onChange={(e) =&gt; setPrompt(e.target.value)}
              placeholder="Ask anything about your code…"
              onKeyDown={(e) =&gt; {
                if (e.key === 'Enter' && !e.shiftKey) {
                  e.preventDefault();
                  if (prompt.trim()) {
                    setActiveAction('generate');
                    runAction('generate', prompt);
                  }
                }
              }}
              disabled={isLoading}
              rows={2}
            /&gt;
            &lt;button
              id="ai-chat-submit"
              type="submit"
              className="ai-submit-btn"
              disabled={isLoading || !prompt.trim()}
              onClick={() =&gt; { setActiveAction('generate'); }}
            &gt;
              &lt;svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor"&gt;
                &lt;path d="M2 21l21-9L2 3v7l15 2-15 2z" /&gt;
              &lt;/svg&gt;
            &lt;/button&gt;
          &lt;/form&gt;
        )}
      &lt;/div&gt;
    &lt;/aside&gt;
  );
}
```

frontend/src\components\Console\Console.css 268 lines

```
.console {
  display: flex;
  flex-direction: column;
  height: 100%;
  background: var(--surface-0);
  overflow: hidden;
}

/* Header */
.console-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 1rem;
  height: 42px;
  border-bottom: 1px solid var(--border);
  background: var(--surface-glass);
  backdrop-filter: blur(10px);
  flex-shrink: 0;
}

.console-tabs {
  display: flex;
  gap: 0.125rem;
}

.console-tab {
  display: flex;
  align-items: center;
  gap: 0.375rem;
  height: 28px;
  padding: 0 0.75rem;
  background: transparent;
  border: none;
  border-radius: 6px;
  color: var(--text-muted);
  font-size: 0.78rem;
  font-weight: 500;
  font-family: inherit;
  cursor: pointer;
  position: relative;
  transition: all 0.15s ease;
}

.console-tab:hover {
  background: var(--surface-2);
  color: var(--text-secondary);
}

.console-tab.active {
```

```
  background: var(--surface-2);
  color: var(--text-primary);
}

.tab-dot {
  width: 6px;
  height: 6px;
  border-radius: 50%;
  flex-shrink: 0;
}

.tab-dot.success { background: #10b981; }
.tab-dot.error { background: #ef4444; }

/* Status Badge */
.console-status {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  font-size: 0.75rem;
  font-weight: 500;
  padding: 3px 10px;
  border-radius: 20px;
  transition: all 0.3s ease;
}

.status-dot {
  width: 7px;
  height: 7px;
  border-radius: 50%;
  flex-shrink: 0;
  transition: background 0.3s ease;
}

.status-meta {
  color: var(--text-muted);
  font-weight: 400;
  font-size: 0.7rem;
  border-left: 1px solid var(--border);
  padding-left: 0.5rem;
  margin-left: 0.5rem;
}

/* Body */
.console-body {
  flex: 1;
  overflow: hidden;
  display: flex;
  flex-direction: column;
}

.console-content {
  flex: 1;
  overflow: auto;
```

```
  height: 100%;
  display: flex;
  flex-direction: column;
}

/* Output text */
.console-output {
  flex: 1;
  margin: 0;
  padding: 1rem;
  font-family: 'JetBrains Mono', 'Fira Code', monospace;
  font-size: 0.825rem;
  line-height: 1.6;
  tab-size: 4;
  white-space: pre-wrap;
  word-break: break-word;
  overflow: auto;
}

.success-output {
  color: var(--text-primary);
}

.term-output {
  white-space: pre-wrap;
  color: var(--text-secondary);
}

.term-input {
  white-space: pre-wrap;
  color: var(--teal);
}

.term-error {
  white-space: pre-wrap;
  color: #f87171;
}

.terminal-input-row {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  margin-top: 0.25rem;
}

.term-prompt {
  color: var(--teal);
  font-weight: bold;
}

.terminal-input {
  background: transparent;
  border: none;
  outline: none;
```

```
  color: var(--text-primary);
  font-family: inherit;
  font-size: inherit;
  flex: 1;
}

.error-output {
  color: #f87171;
  background: rgba(239, 68, 68, 0.05);
}

/* States */
.console-placeholder {
  flex: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 0.5rem;
  color: var(--text-muted);
  padding: 2rem;
  text-align: center;
}

.console-placeholder svg {
  opacity: 0.4;
  margin-bottom: 0.5rem;
}

.console-placeholder p {
  margin: 0;
  font-size: 0.875rem;
  color: var(--text-secondary);
}

.console-placeholder span {
  font-size: 0.775rem;
}

.console-empty {
  flex: 1;
  display: flex;
  align-items: center;
  justify-content: center;
  color: var(--text-muted);
  font-size: 0.825rem;
  gap: 0.5rem;
}

.console-running {
  flex: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
```

```
  justify-content: center;
  gap: 1rem;
  color: var(--text-secondary);
  font-size: 0.875rem;
}

.console-spinner {
  width: 28px;
  height: 28px;
  border: 3px solid var(--border);
  border-top-color: var(--teal);
  border-radius: 50%;
  animation: spin 0.7s linear infinite;
}

@keyframes spin {
  to { transform: rotate(360deg); }
}

/* Stdin */
.stdin-textarea {
  flex: 1;
  width: 100%;
  height: 100%;
  resize: none;
  background: transparent;
  border: none;
  color: var(--text-primary);
  font-family: 'JetBrains Mono', monospace;
  font-size: 0.825rem;
  line-height: 1.6;
  padding: 1rem;
  outline: none;
  box-sizing: border-box;
}

.stdin-textarea::placeholder {
  color: var(--text-muted);
  font-style: italic;
}

/* Stop button */
.stop-btn {
  background: rgba(239, 68, 68, 0.1);
  color: #ef4444;
  border: none;
  border-radius: 4px;
  padding: 2px 6px;
  font-size: 0.7rem;
  font-family: inherit;
  cursor: pointer;
}
.stop-btn:hover {
```

```
  background: rgba(239, 68, 68, 0.2);
}
```

frontend/src\components\Console\Console.jsx 159 lines

```
import React, { useState } from 'react';
import './Console.css';

const STATUS_CONFIG = {
  idle: { label: 'Ready', color: 'var(--text-muted)', bg: 'transparent', dot:
'#6b7280' },
  running: { label: 'Running…', color: '#f59e0b', bg: 'rgba(245,158,11,0.1)', dot:
'#f59e0b' },
  success: { label: 'Accepted', color: '#10b981', bg: 'rgba(16,185,129,0.1)', dot:
'#10b981' },
  error: { label: 'Error', color: '#ef4444', bg: 'rgba(239,68,68,0.1)', dot:
'#ef4444' },
  tle: { label: 'Time Limit Exceeded', color: '#f59e0b', bg: 'rgba(245,158,11,0.1)',
dot: '#f59e0b' },
  mle: { label: 'Memory Limit Exceeded', color: '#f59e0b', bg:
'rgba(245,158,11,0.1)', dot: '#f59e0b' },
  compile_error: { label: 'Compilation Error', color: '#ef4444', bg:
'rgba(239,68,68,0.1)', dot: '#ef4444' },
};

const getStatusFromResult = (result) =&gt; {
  if (!result) return 'idle';
  const id = result.status?.id;
  if (id === 3) return 'success';
  if (id === 6) return 'compile_error';
  if (id === 5) return 'tle';
  if (id === 7) return 'mle';
  if (id &gt;= 4) return 'error';
  return 'idle';
};

export default function Console({ terminalData = [], result, isRunning,
isInteractive, onSendInput, onStop }) {
  const [activeTab, setActiveTab] = useState('output');
  const [inputValue, setInputValue] = useState('');

  const status = isRunning ? 'running' : getStatusFromResult(result);
  const statusConfig = STATUS_CONFIG[status];

  const hasOutput = terminalData.length &gt; 0;
  const hasError = terminalData.some(d =&gt; d.type === 'error') || (result &&
(result.stderr || result.compile_output));

  const errorText = terminalData.filter(d =&gt; d.type === 'error').map(d =&gt;
d.text).join('') + (result?.compile_output || result?.stderr || '');

  const handleKeyDown = (e) =&gt; {
    if (e.key === 'Enter') {
      e.preventDefault();
      onSendInput(inputValue);
```

```
      setInputValue('');
    }
  };

  return (
    &lt;div className="console"&gt;
      {/* Console Header */}
      &lt;div className="console-header"&gt;
        &lt;div className="console-tabs"&gt;

          &lt;button
            id="tab-output"
            className={`console-tab ${activeTab === 'output' ? 'active' : ''}`}
            onClick={() =&gt; setActiveTab('output')}
          &gt;
            &lt;svg width="12" height="12" viewBox="0 0 24 24"
fill="currentColor"&gt;
              &lt;path d="M20 4H4c-1.1 0-2 .9-2 2v12c0 1.1.9 2 2 2h16c1.1 0 2-.9 2-
2V6c0-1.1-.9-2-2-2zm0 14H4V6h16v12zM6 10h2v2H6zm0 4h8v2H6zm10 0h2v2h-2zm-6-4h8v2h-
8z" /&gt;
            &lt;/svg&gt;
            Terminal
            {hasOutput && &lt;span className="tab-dot success" /&gt;}
          &lt;/button&gt;
          &lt;button
            id="tab-errors"
            className={`console-tab ${activeTab === 'errors' ? 'active' : ''}`}
            onClick={() =&gt; setActiveTab('errors')}
          &gt;
            &lt;svg width="12" height="12" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2"&gt;
              &lt;circle cx="12" cy="12" r="10" /&gt;
              &lt;line x1="12" y1="8" x2="12" y2="12" /&gt;
              &lt;line x1="12" y1="16" x2="12.01" y2="16" strokeWidth="3"
strokeLinecap="round" /&gt;
            &lt;/svg&gt;
            Errors
            {hasError && &lt;span className="tab-dot error" /&gt;}
          &lt;/button&gt;
        &lt;/div&gt;

        {/* Status Badge */}
        &lt;div className="console-status" style={{ color: statusConfig.color,
background: statusConfig.bg }}&gt;
          {isRunning && (
            &lt;button className="stop-btn" onClick={onStop} title="Force Stop
Execution"&gt;
              ■ Stop
            &lt;/button&gt;
          )}
          &lt;span className="status-dot" style={{ background: statusConfig.dot }}
/&gt;
          &lt;span&gt;{statusConfig.label}&lt;/span&gt;
          {result?.time && status === 'success' && (
```

```
            &lt;span className="status-meta"&gt;
              {result.time}s · {result.memory ? `${(result.memory /
1024).toFixed(1)} MB` : ''}
              {result.mock && ' · Demo'}
            &lt;/span&gt;
          )}
        &lt;/div&gt;
      &lt;/div&gt;

      {/* Console Body */}
      &lt;div className="console-body"&gt;
        {/* Output Tab (Terminal) */}
        {activeTab === 'output' && (
          &lt;div className="console-content terminal-mode"&gt;
            {hasOutput || isInteractive ? (
              &lt;div className="console-output success-output"&gt;
                {terminalData.map((d, i) =&gt; (
                  &lt;span key={i} className={`term-${d.type}`}&gt;
{d.text}&lt;/span&gt;
                ))}
                {isInteractive && (
                  &lt;div className="terminal-input-row"&gt;
                    &lt;span className="term-prompt"&gt;❯&lt;/span&gt;
                    &lt;input
                      type="text"
                      className="terminal-input"
                      value={inputValue}
                      onChange={(e) =&gt; setInputValue(e.target.value)}
                      onKeyDown={handleKeyDown}
                      placeholder="Type input and press Enter..."
                      autoComplete="off"
                      autoFocus
                    /&gt;
                  &lt;/div&gt;
                )}
              &lt;/div&gt;
            ) : result && !hasOutput && !hasError ? (
              &lt;div className="console-empty"&gt;
                &lt;span&gt;✓ Program exited with no output&lt;/span&gt;
              &lt;/div&gt;
            ) : (
              &lt;div className="console-placeholder"&gt;
                &lt;svg width="32" height="32" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5"&gt;
                  &lt;polyline points="4 17 10 11 4 5" /&gt;
                  &lt;line x1="12" y1="19" x2="20" y2="19" /&gt;
                &lt;/svg&gt;
                &lt;p&gt;Click &lt;strong&gt;Run&lt;/strong&gt; to execute your
code&lt;/p&gt;
                &lt;span&gt;Output will stream here magically ✨&lt;/span&gt;
              &lt;/div&gt;
            )}
          &lt;/div&gt;
        )}
```

```
        {/* Errors Tab */}
        {activeTab === 'errors' && (
          &lt;div className="console-content"&gt;
            {hasError ? (
              &lt;pre className="console-output error-output"&gt;
{errorText}&lt;/pre&gt;
            ) : (
              &lt;div className="console-placeholder"&gt;
                &lt;svg width="32" height="32" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5"&gt;
                  &lt;path d="M22 11.08V12a10 10 0 1 1-5.93-9.14" /&gt;
                  &lt;polyline points="22 4 12 14.01 9 11.01" /&gt;
                &lt;/svg&gt;
                &lt;p&gt;No errors detected&lt;/p&gt;
                &lt;span&gt;Compilation and runtime errors will appear
here&lt;/span&gt;
              &lt;/div&gt;
            )}
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

frontend/src\components\Editor\CodeEditor.css 32 lines

```
.code-editor-wrapper {
  height: 100%;
  width: 100%;
  overflow: hidden;
  border-radius: 0;
}

.editor-loading {
  height: 100%;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 1rem;
  color: var(--text-muted);
  font-size: 0.875rem;
  background: var(--surface-0);
}

.editor-loading-spinner {
  width: 32px;
  height: 32px;
  border: 3px solid var(--border);
  border-top-color: var(--accent);
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
}

@keyframes spin {
  to { transform: rotate(360deg); }
}
```

frontend/src\components\Editor\CodeEditor.jsx 133 lines

```
import React, { useRef, useEffect } from 'react';
import MonacoEditor from '@monaco-editor/react';
import './CodeEditor.css';

// Custom dark theme definition
const defineTheme = (monaco) =&gt; {
  monaco.editor.defineTheme('compilex-dark', {
    base: 'vs-dark',
    inherit: true,
    rules: [
      { token: 'comment', foreground: '6a7384', fontStyle: 'italic' },
      { token: 'keyword', foreground: 'c792ea', fontStyle: 'bold' },
      { token: 'string', foreground: 'c3e88d' },
      { token: 'number', foreground: 'f78c6c' },
      { token: 'type', foreground: '82aaff' },
      { token: 'function', foreground: '82aaff' },
      { token: 'variable', foreground: 'eeffff' },
      { token: 'operator', foreground: '89ddff' },
      { token: 'delimiter', foreground: '89ddff' },
    ],
    colors: {
      'editor.background': '#0d0d1a',
      'editor.foreground': '#e2e8f0',
      'editor.lineHighlightBackground': '#1a1a2e',
      'editor.selectionBackground': '#7c3aed33',
      'editor.inactiveSelectionBackground': '#7c3aed1a',
      'editorLineNumber.foreground': '#3d4461',
      'editorLineNumber.activeForeground': '#7c3aed',
      'editorCursor.foreground': '#06d6a0',
      'editorGutter.background': '#0d0d1a',
      'editorWidget.background': '#1a1a2e',
      'editorWidget.border': '#2d2d4e',
      'editorSuggestWidget.background': '#1a1a2e',
      'editorSuggestWidget.border': '#2d2d4e',
      'editorSuggestWidget.selectedBackground': '#7c3aed22',
      'input.background': '#0d0d1a',
      'focusBorder': '#7c3aed',
      'scrollbarSlider.background': '#2d2d4e99',
      'scrollbarSlider.hoverBackground': '#7c3aed44',
    },
  });

  monaco.editor.defineTheme('compilex-light', {
    base: 'vs',
    inherit: true,
    rules: [
      { token: 'comment', foreground: '6a7384', fontStyle: 'italic' },
      { token: 'keyword', foreground: '7c3aed', fontStyle: 'bold' },
      { token: 'string', foreground: '059669' },
      { token: 'number', foreground: 'e11d48' },
```

```
    ],
    colors: {
      'editor.background': '#f8fafc',
      'editor.foreground': '#1e293b',
      'editor.lineHighlightBackground': '#f1f5f9',
      'editor.selectionBackground': '#7c3aed22',
      'editorLineNumber.foreground': '#94a3b8',
      'editorLineNumber.activeForeground': '#7c3aed',
      'editorCursor.foreground': '#7c3aed',
      'editorGutter.background': '#f8fafc',
    },
  });
};

export default function CodeEditor({ value, language, theme, onChange }) {
  const editorRef = useRef(null);
  const monacoRef = useRef(null);

  const handleEditorMount = (editor, monaco) =&gt; {
    editorRef.current = editor;
    monacoRef.current = monaco;
    defineTheme(monaco);
    monaco.editor.setTheme(theme === 'dark' ? 'compilex-dark' : 'compilex-light');

    // Custom font
    editor.updateOptions({ fontFamily: "'JetBrains Mono', 'Fira Code', 'Cascadia
Code', monospace" });
  };

  // Update theme when it changes
  useEffect(() =&gt; {
    if (monacoRef.current) {
      monacoRef.current.editor.setTheme(theme === 'dark' ? 'compilex-dark' :
'compilex-light');
    }
  }, [theme]);

  const editorOptions = {
    fontSize: 14,
    lineHeight: 22,
    minimap: { enabled: false },
    scrollBeyondLastLine: false,
    automaticLayout: true,
    tabSize: 4,
    insertSpaces: true,
    wordWrap: 'on',
    renderLineHighlight: 'all',
    cursorStyle: 'line',
    cursorBlinking: 'smooth',
    smoothScrolling: true,
    contextmenu: true,
    selectOnLineNumbers: true,
    roundedSelection: true,
    bracketPairColorization: { enabled: true },
```

```
    padding: { top: 16, bottom: 16 },
    lineNumbers: 'on',
    glyphMargin: false,
    folding: true,
    showFoldingControls: 'mouseover',
    links: true,
    colorDecorators: true,
    'semanticHighlighting.enabled': true,
  };

  return (
    &lt;div className="code-editor-wrapper"&gt;
      &lt;MonacoEditor
        height="100%"
        language={language}
        value={value}
        theme={theme === 'dark' ? 'compilex-dark' : 'compilex-light'}
        options={editorOptions}
        onMount={handleEditorMount}
        onChange={onChange}
        loading={
          &lt;div className="editor-loading"&gt;
            &lt;div className="editor-loading-spinner" /&gt;
            &lt;span&gt;Loading Editor…&lt;/span&gt;
          &lt;/div&gt;
        }
      /&gt;
    &lt;/div&gt;
  );
}
```

frontend/src\components\Navbar\Navbar.css 226 lines

```
.navbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  height: 56px;
  padding: 0 1.25rem;
  background: var(--surface-glass);
  backdrop-filter: blur(20px);
  -webkit-backdrop-filter: blur(20px);
  border-bottom: 1px solid var(--border);
  position: relative;
  z-index: 100;
  flex-shrink: 0;
}

/* Logo */
.navbar-logo {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  user-select: none;
}

.logo-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 32px;
  height: 32px;
  background: var(--surface-2);
  border-radius: 8px;
  border: 1px solid var(--border);
}

.logo-text {
  font-size: 1.2rem;
  font-weight: 700;
  letter-spacing: -0.02em;
  color: var(--text-primary);
}

.logo-accent {
  background: linear-gradient(135deg, var(--accent), var(--teal));
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}

.logo-badge {
  font-size: 0.6rem;
```

```
  font-weight: 700;
  letter-spacing: 0.08em;
  color: var(--accent);
  background: rgba(124, 58, 237, 0.15);
  border: 1px solid rgba(124, 58, 237, 0.3);
  border-radius: 4px;
  padding: 1px 5px;
  text-transform: uppercase;
}

/* Center */
.navbar-center {
  display: flex;
  align-items: center;
}

.language-selector-wrapper {
  position: relative;
  display: flex;
  align-items: center;
  background: var(--surface-2);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 0 0.75rem;
  height: 36px;
  gap: 0.5rem;
  transition: border-color 0.2s, box-shadow 0.2s;
}

.language-selector-wrapper:hover {
  border-color: var(--accent);
  box-shadow: 0 0 0 3px rgba(124, 58, 237, 0.1);
}

.language-icon {
  font-size: 1rem;
  line-height: 1;
  pointer-events: none;
}

.language-selector {
  background: transparent;
  border: none;
  color: var(--text-primary);
  font-size: 0.875rem;
  font-weight: 500;
  font-family: inherit;
  cursor: pointer;
  outline: none;
  appearance: none;
  -webkit-appearance: none;
  padding-right: 1.25rem;
  min-width: 110px;
}
```

```
.language-selector option {
  background: var(--surface-1);
  color: var(--text-primary);
}

.select-chevron {
  position: absolute;
  right: 0.75rem;
  pointer-events: none;
  color: var(--text-muted);
}

/* Actions */
.navbar-actions {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.btn-icon {
  display: flex;
  align-items: center;
  gap: 0.35rem;
  height: 36px;
  padding: 0 0.75rem;
  background: var(--surface-2);
  border: 1px solid var(--border);
  border-radius: 8px;
  color: var(--text-secondary);
  cursor: pointer;
  font-size: 0.8rem;
  font-weight: 500;
  font-family: inherit;
  transition: all 0.2s ease;
}

.btn-icon:hover {
  background: var(--surface-3);
  border-color: var(--border-hover);
  color: var(--text-primary);
}

.btn-ai {
  color: var(--accent-light);
  border-color: rgba(124, 58, 237, 0.3);
}

.btn-ai:hover,
.btn-ai.active {
  background: rgba(124, 58, 237, 0.15);
  border-color: var(--accent);
  color: var(--accent-light);
  box-shadow: 0 0 12px rgba(124, 58, 237, 0.2);
```

```
}

.btn-run {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  height: 36px;
  padding: 0 1.25rem;
  background: linear-gradient(135deg, #059669, #06d6a0);
  border: none;
  border-radius: 8px;
  color: #fff;
  font-size: 0.875rem;
  font-weight: 600;
  font-family: inherit;
  cursor: pointer;
  transition: all 0.2s ease;
  box-shadow: 0 2px 12px rgba(6, 214, 160, 0.3);
  position: relative;
  overflow: hidden;
}

.btn-run::before {
  content: '';
  position: absolute;
  inset: 0;
  background: rgba(255, 255, 255, 0.1);
  opacity: 0;
  transition: opacity 0.2s;
}

.btn-run:hover::before {
  opacity: 1;
}

.btn-run:hover {
  transform: translateY(-1px);
  box-shadow: 0 4px 20px rgba(6, 214, 160, 0.4);
}

.btn-run:active {
  transform: translateY(0);
}

.btn-run:disabled {
  opacity: 0.8;
  cursor: not-allowed;
  transform: none;
}

.btn-run.running {
  background: linear-gradient(135deg, #047857, #059669);
}
```

```
/* Spinner */
.spinner {
  width: 14px;
  height: 14px;
  border: 2px solid rgba(255, 255, 255, 0.3);
  border-top-color: #fff;
  border-radius: 50%;
  animation: spin 0.7s linear infinite;
}

@keyframes spin {
  to { transform: rotate(360deg); }
}
```

frontend/src\components\Navbar\Navbar.jsx 146 lines

```
import React from 'react';
import { LANGUAGES } from '../../constants/languages';
import './Navbar.css';

const RunIcon = () =&gt; (
  &lt;svg width="16" height="16" viewBox="0 0 24 24" fill="currentColor"&gt;
    &lt;path d="M8 5v14l11-7z" /&gt;
  &lt;/svg&gt;
);

const MoonIcon = () =&gt; (
  &lt;svg width="16" height="16" viewBox="0 0 24 24" fill="currentColor"&gt;
    &lt;path d="M21 12.79A9 9 0 1 1 11.21 3 7 7 0 0 0 21 12.79z" /&gt;
  &lt;/svg&gt;
);

const SunIcon = () =&gt; (
  &lt;svg width="16" height="16" viewBox="0 0 24 24" fill="currentColor"&gt;
    &lt;circle cx="12" cy="12" r="5" /&gt;
    &lt;path d="M12 1v2M12 21v2M4.22 4.22l1.42 1.42M18.36 18.36l1.42 1.42M1 12h2M21
12h2M4.22 19.78l1.42-1.42M18.36 5.64l1.42-1.42" stroke="currentColor"
strokeWidth="2" fill="none" /&gt;
  &lt;/svg&gt;
);

const ShareIcon = () =&gt; (
  &lt;svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2"&gt;
    &lt;circle cx="18" cy="5" r="3" /&gt;&lt;circle cx="6" cy="12" r="3"
/&gt;&lt;circle cx="18" cy="19" r="3" /&gt;
    &lt;path d="M8.59 13.51l6.83 3.98M15.41 6.51l-6.82 3.98" /&gt;
  &lt;/svg&gt;
);

const AIIcon = () =&gt; (
  &lt;svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2"&gt;
    &lt;path d="M12 2a10 10 0 1 0 10 10A10 10 0 0 0 12 2z" /&gt;
    &lt;path d="M8 14s1.5 2 4 2 4-2 4-2" /&gt;
    &lt;line x1="9" y1="9" x2="9.01" y2="9" strokeWidth="3" strokeLinecap="round"
/&gt;
    &lt;line x1="15" y1="9" x2="15.01" y2="9" strokeWidth="3" strokeLinecap="round"
/&gt;
  &lt;/svg&gt;
);

export default function Navbar({
  selectedLanguage,
  onLanguageChange,
  onRun,
```

```
  isRunning,
  theme,
  onThemeToggle,
  onAIToggle,
  aiPanelOpen,
}) {
  const lang = LANGUAGES.find((l) =&gt; l.id === selectedLanguage) || LANGUAGES[0];

  return (
    &lt;nav className="navbar"&gt;
      {/* Logo */}
      &lt;div className="navbar-logo"&gt;
        &lt;div className="logo-icon"&gt;
          &lt;svg width="22" height="22" viewBox="0 0 24 24" fill="none"&gt;
            &lt;path d="M13 2L3 14h9l-1 8 10-12h-9l1-8z" fill="url(#bolt)" /&gt;
            &lt;defs&gt;
              &lt;linearGradient id="bolt" x1="0%" y1="0%" x2="100%" y2="100%"&gt;
                &lt;stop offset="0%" stopColor="#7c3aed" /&gt;
                &lt;stop offset="100%" stopColor="#06d6a0" /&gt;
              &lt;/linearGradient&gt;
            &lt;/defs&gt;
          &lt;/svg&gt;
        &lt;/div&gt;
        &lt;span className="logo-text"&gt;
          Compile&lt;span className="logo-accent"&gt;X&lt;/span&gt;
        &lt;/span&gt;
        &lt;span className="logo-badge"&gt;AI&lt;/span&gt;
      &lt;/div&gt;

      {/* Center: Language Selector */}
      &lt;div className="navbar-center"&gt;
        &lt;div className="language-selector-wrapper"&gt;
          &lt;span className="language-icon"&gt;{lang.icon}&lt;/span&gt;
          &lt;select
            id="language-select"
            className="language-selector"
            value={selectedLanguage}
            onChange={(e) =&gt; onLanguageChange(e.target.value)}
          &gt;
            {LANGUAGES.map((l) =&gt; (
              &lt;option key={l.id} value={l.id}&gt;
                {l.label}
              &lt;/option&gt;
            ))}
          &lt;/select&gt;
          &lt;svg className="select-chevron" width="12" height="12" viewBox="0 0 24
24" fill="currentColor"&gt;
            &lt;path d="M6 9l6 6 6-6" stroke="currentColor" strokeWidth="2"
fill="none" /&gt;
          &lt;/svg&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      {/* Right: Actions */}
```

```
      &lt;div className="navbar-actions"&gt;
        {/* AI Toggle */}
        &lt;button
          id="ai-panel-toggle"
          className={`btn-icon btn-ai ${aiPanelOpen ? 'active' : ''}`}
          onClick={onAIToggle}
          title="Toggle AI Assistant"
        &gt;
          &lt;AIIcon /&gt;
          &lt;span&gt;AI&lt;/span&gt;
        &lt;/button&gt;

        {/* Theme Toggle */}
        &lt;button
          id="theme-toggle"
          className="btn-icon"
          onClick={onThemeToggle}
          title={`Switch to ${theme === 'dark' ? 'light' : 'dark'} mode`}
        &gt;
          {theme === 'dark' ? &lt;SunIcon /&gt; : &lt;MoonIcon /&gt;}
        &lt;/button&gt;

        {/* Share */}
        &lt;button id="share-btn" className="btn-icon" title="Share code"&gt;
          &lt;ShareIcon /&gt;
        &lt;/button&gt;

        {/* Run Button */}
        &lt;button
          id="run-btn"
          className={`btn-run ${isRunning ? 'running' : ''}`}
          onClick={onRun}
          disabled={isRunning}
        &gt;
          {isRunning ? (
            &lt;&gt;
              &lt;div className="spinner" /&gt;
              &lt;span&gt;Running…&lt;/span&gt;
            &lt;/&gt;
          ) : (
            &lt;&gt;
              &lt;RunIcon /&gt;
              &lt;span&gt;Run&lt;/span&gt;
            &lt;/&gt;
          )}
        &lt;/button&gt;
      &lt;/div&gt;
    &lt;/nav&gt;
  );
}
```

frontend/src\components\StatusBar\StatusBar.css 71 lines

```
.status-bar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  height: 24px;
  padding: 0 0.75rem;
  background: var(--accent);
  color: rgba(255, 255, 255, 0.85);
  font-size: 0.7rem;
  font-weight: 500;
  letter-spacing: 0.02em;
  flex-shrink: 0;
  user-select: none;
}

.status-left,
.status-right {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.status-item {
  display: flex;
  align-items: center;
  gap: 0.3rem;
  opacity: 0.85;
  transition: opacity 0.15s;
}

.status-item:hover { opacity: 1; }

.status-separator {
  width: 1px;
  height: 12px;
  background: rgba(255, 255, 255, 0.3);
}

.demo-badge {
  display: flex;
  align-items: center;
  gap: 0.35rem;
  background: rgba(255, 255, 255, 0.15);
  border-radius: 10px;
  padding: 1px 8px;
  opacity: 1 !important;
  font-weight: 600;
}

.demo-dot {
```

```
  width: 5px;
  height: 5px;
  border-radius: 50%;
  background: #fbbf24;
  animation: pulse 2s ease-in-out infinite;
}

@keyframes pulse {
  0%, 100% { opacity: 1; transform: scale(1); }
  50% { opacity: 0.5; transform: scale(0.8); }
}

.lang-badge {
  background: rgba(255, 255, 255, 0.15);
  border-radius: 3px;
  padding: 0 5px;
  font-size: 0.65rem;
  letter-spacing: 0.05em;
  opacity: 1 !important;
}
```

frontend/src\components\StatusBar\StatusBar.jsx 38 lines

```
import React from 'react';
import { isMockMode } from '../../services/judge0';
import './StatusBar.css';

export default function StatusBar({ selectedLanguage, linesCount, charCount }) {
  const mock = isMockMode();

  return (
    &lt;div className="status-bar"&gt;
      &lt;div className="status-left"&gt;
        {mock && (
          &lt;span className="status-item demo-badge" title="Add API keys to .env
for live execution"&gt;
            &lt;span className="demo-dot" /&gt;
            Demo Mode
          &lt;/span&gt;
        )}
        &lt;span className="status-item"&gt;
          &lt;svg width="10" height="10" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2"&gt;
            &lt;polyline points="4 17 10 11 4 5" /&gt;
            &lt;line x1="12" y1="19" x2="20" y2="19" /&gt;
          &lt;/svg&gt;
          CompileX IDE
        &lt;/span&gt;
      &lt;/div&gt;

      &lt;div className="status-right"&gt;
        &lt;span className="status-item"&gt;Ln {linesCount}&lt;/span&gt;
        &lt;span className="status-separator" /&gt;
        &lt;span className="status-item"&gt;{charCount} chars&lt;/span&gt;
        &lt;span className="status-separator" /&gt;
        &lt;span className="status-item"&gt;UTF-8&lt;/span&gt;
        &lt;span className="status-separator" /&gt;
        &lt;span className="status-item lang-badge"&gt;
{selectedLanguage?.toUpperCase()}&lt;/span&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

frontend/src\constants\languages.js 132 lines

```
// Language configurations with Judge0 CE language IDs
// Judge0 IDs: https://ce.judge0.com/languages/

export const LANGUAGES = [
  {
    id: 'python',
    label: 'Python 3',
    judge0Id: 71,
    monacoId: 'python',
    icon: '🐍',
    color: '#3b82f6',
    defaultCode: `# Python 3 - CompileX
def greet(name):
    return f"Hello, {name}! Welcome to CompileX 🚀"

name = input("Enter your name: ") if False else "World"
print(greet(name))

# Quick demo
for i in range(1, 6):
    print(f"  {'★' * i}")
`,
  },
  {
    id: 'javascript',
    label: 'JavaScript',
    judge0Id: 63,
    monacoId: 'javascript',
    icon: '⚡',
    color: '#f59e0b',
    defaultCode: `// JavaScript - CompileX
function greet(name) {
  return \`Hello, \${name}! Welcome to CompileX 🚀\`;
}

const name = "World";
console.log(greet(name));

// Quick demo
const stars = Array.from({ length: 5 }, (_, i) =&gt; '★'.repeat(i + 1));
stars.forEach(s =&gt; console.log(' ' + s));
`,
  },
  {
    id: 'cpp',
    label: 'C++',
    judge0Id: 54,
    monacoId: 'cpp',
    icon: '⚙️',
    color: '#8b5cf6',
```

```
    defaultCode: `// C++ - CompileX
#include &lt;iostream&gt;
#include &lt;string&gt;
using namespace std;

string greet(string name) {
    return "Hello, " + name + "! Welcome to CompileX 🚀";
}

int main() {
    string name = "World";
    cout &lt;&lt; greet(name) &lt;&lt; endl;

    // Quick demo
    for (int i = 1; i &lt;= 5; i++) {
        cout &lt;&lt; "  ";
        for (int j = 0; j &lt; i; j++) cout &lt;&lt; "*";
        cout &lt;&lt; endl;
    }
    return 0;
}
`,
  },
  {
    id: 'c',
    label: 'C',
    judge0Id: 50,
    monacoId: 'c',
    icon: '🔧',
    color: '#06b6d4',
    defaultCode: `// C - CompileX
#include &lt;stdio.h&gt;
#include &lt;string.h&gt;

void greet(const char* name) {
    printf("Hello, %s! Welcome to CompileX \\n", name);
}

int main() {
    greet("World");

    // Quick demo
    for (int i = 1; i &lt;= 5; i++) {
        printf("  ");
        for (int j = 0; j &lt; i; j++) printf("*");
        printf("\\n");
    }
    return 0;
}
`,
  },
  {
    id: 'java',
    label: 'Java',
```

```
    judge0Id: 62,
    monacoId: 'java',
    icon: '☕',
    color: '#ef4444',
    defaultCode: `// Java - CompileX
public class Main {
    static String greet(String name) {
        return "Hello, " + name + "! Welcome to CompileX 🚀";
    }

    public static void main(String[] args) {
        System.out.println(greet("World"));

        // Quick demo
        for (int i = 1; i &lt;= 5; i++) {
            System.out.print("  ");
            for (int j = 0; j &lt; i; j++) System.out.print("*");
            System.out.println();
        }
    }
}
`,
  },
];

export const getLanguageById = (id) =&gt; LANGUAGES.find((l) =&gt; l.id === id) ||
LANGUAGES[0];
export const getLanguageByJudge0Id = (j0id) =&gt; LANGUAGES.find((l) =&gt;
l.judge0Id === j0id);
```

frontend/src\index.css 6 lines

```
/* Reset index.css — App.css handles everything */
*, *::before, *::after {
  box-sizing: border-box;
}
body { margin: 0; }
```

frontend/src\main.jsx 11 lines

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  &lt;StrictMode&gt;
    &lt;App /&gt;
  &lt;/StrictMode&gt;,
)
```

📄 **frontend/src\services\gemini.js** 158 lines

```
// Free AI Service using Groq (Llama-3)
// Get a fully free API key without a credit card from:
https://console.groq.com/keys

const GROQ_API_KEY = import.meta.env.VITE_GROQ_API_KEY ||
import.meta.env.VITE_GEMINI_API_KEY;
const GROQ_BASE_URL = 'https://api.groq.com/openai/v1/chat/completions';

const MOCK_MODE = !GROQ_API_KEY;

// ─── Mock AI Responses ──────────────────────────────────────────────────
// Note: We use string concatenation to avoid nested template literal issues
const F = '```'; // fence shorthand

const MOCK_RESPONSES = {
  generate: (prompt, language) =&gt; ({
    text: [
      'Here\'s a ' + language + ' implementation for: **' + prompt + '**',
      '',
      F + language,
      '# Demo mode - Add VITE_GROQ_API_KEY to .env for real AI responses',
      '',
      'def binary_search(arr, target):',
      '    return -1',
      F
    ].join('\n'),
    type: 'code',
  }),
  debug: (code, error) =&gt; ({
    text: '## 🔍 Bug Analysis\n**Demo Mode** — Add VITE_GROQ_API_KEY to .env.',
    type: 'text',
  }),
  explain: (code) =&gt; ({
    text: '## 📖 Code Explanation\n**Demo Mode** — Add VITE_GROQ_API_KEY to .env.',
    type: 'text',
  }),
  optimize: () =&gt; ({
    text: '## ⚡ Optimization\n**Demo Mode** — Add VITE_GROQ_API_KEY to .env.',
    type: 'text',
  }),
  convert: (fromLang, toLang) =&gt; ({
    text: '## 🔄 Language Conversion\n**Demo Mode** — Add VITE_GROQ_API_KEY.',
    type: 'code',
  }),
};

// Simulate delay
const delay = (ms) =&gt; new Promise((res) =&gt; setTimeout(res, ms));

// ─── Real Groq (Llama) API Call ─────────────────────────────────────────
```

```
const callAI = async (prompt) =&gt; {
  const response = await fetch(GROQ_BASE_URL, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${GROQ_API_KEY}`
    },
    body: JSON.stringify({
      model: 'llama-3.3-70b-versatile',
      messages: [{ role: 'user', content: prompt }],
      temperature: 0.7,
      max_tokens: 2048,
    }),
  });
  if (!response.ok) {
    const err = await response.json();
    throw new Error(err?.error?.message || 'AI API error: ' + response.statusText);
  }
  const data = await response.json();
  return data.choices?.[0]?.message?.content || 'No response from AI.';
};

// ─── Formatting Helpers ──────────────────────────────────────────────────────
const stringifyContext = (contextBundle, language) =&gt; {
  if (typeof contextBundle === 'string') {
    return `${F}${language}\n${contextBundle}\n${F}`;
  }
  if (!Array.isArray(contextBundle)) return '';
  return contextBundle.map(f =&gt; `**File:
${f.name}**\n${F}${language}\n${f.content || ''}\n${F}`).join('\n\n');
};

// ─── Public AI Feature APIs ──────────────────────────────────────────────────
export const generateCode = async (prompt, language) =&gt; {
  if (MOCK_MODE) {
    await delay(1200);
    return MOCK_RESPONSES.generate(prompt, language);
  }
  const fullPrompt =
    'You are an expert ' + language + ' programmer. Generate clean, well-commented '
+ language +
    ' code for the following request. Include a brief explanation after the code
block.\n\nRequest: ' +
    prompt + '\n\nProvide the code in a markdown code block with proper syntax
highlighting.';
  const text = await callAI(fullPrompt);
  return { text, type: 'code' };
};

export const debugCode = async (contextBundle, error, language) =&gt; {
  if (MOCK_MODE) {
    await delay(1000);
    return MOCK_RESPONSES.debug(contextBundle, error);
  }
```

```
  const formattedCode = stringifyContext(contextBundle, language);
  const fullPrompt =
    'You are a strict ' + language + ' debugging expert system. Your task is to act
exclusively as a debugger.\n' +
    'Carefully analyze the complete multi-file project and the crash runtime error
output to detect the exact fault.\n\n' +
    '**Project Context:**\n' + formattedCode + '\n\n' +
    '**Error Output:**\n' + F + '\n' + (error || 'No explicitly reported runtime
error. Check for hidden logical bugs.') + '\n' + F + '\n\n' +
    'You MUST structure your response strictly as:\n' +
    '1) **Error Type & Root Cause** (Explain exactly why it crashed)\n' +
    '2) **Fixed Code** (Output the specific rewritten code blocks for the files that
required modifications)\n' +
    '3) **Explanation of the fix**\n';
  const text = await callAI(fullPrompt);
  return { text, type: 'code' };
};

export const explainCode = async (contextBundle, language) =&gt; {
  if (MOCK_MODE) {
    await delay(1000);
    return MOCK_RESPONSES.explain(contextBundle);
  }
  const formattedCode = stringifyContext(contextBundle, language);
  const fullPrompt =
    'You are a patient programming teacher. Explain this ' + language +
    ' project clearly for a beginner. Discuss the architecture across files if
applicable, then break it down line by line or section by section. Use simple
language and emojis.\n\n' +
    formattedCode;
  const text = await callAI(fullPrompt);
  return { text, type: 'text' };
};

export const optimizeCode = async (contextBundle, language) =&gt; {
  if (MOCK_MODE) {
    await delay(1000);
    return MOCK_RESPONSES.optimize();
  }
  const formattedCode = stringifyContext(contextBundle, language);
  const fullPrompt =
    'You are a ' + language + ' performance expert. Analyze this project code for
inefficiencies and provide ' +
    'optimized versions of the necessary files with explanation. Focus on: time
complexity, space complexity, and architectural best practices.\n\n' +
    formattedCode;
  const text = await callAI(fullPrompt);
  return { text, type: 'code' };
};

export const convertCode = async (contextBundle, fromLang, toLang) =&gt; {
  if (MOCK_MODE) {
    await delay(1200);
    return MOCK_RESPONSES.convert(fromLang, toLang);
```

```
  }
  const formattedCode = stringifyContext(contextBundle, fromLang);
  const fullPrompt =
    'Convert this ' + fromLang + ' project to ' + toLang +
    '. Maintain the exact same logic and functionality. Use idiomatic ' + toLang +
    ' modular patterns. Include comments explaining significant translations.\n\n' +
    '**' + fromLang + ' Project Context:**\n' + formattedCode +
    '\n\nProvide the complete ' + toLang + ' equivalents cleanly separated in code
blocks.';
  const text = await callAI(fullPrompt);
  return { text, type: 'code' };
};

export const isAIMockMode = () =&gt; MOCK_MODE;
```

frontend/src\services\judge0.js 41 lines

```
// Custom Local Execution Service
// Replaces Judge0 CE to run natively on localhost:5000

const LOCAL_BACKEND_URL = 'http://localhost:5000';

export const executeCode = async (files, mainFile, languageId, stdin = '') =&gt; {
  // We use base64 encoding just like Judge0 to avoid special character escaping
issues in JSON
  const encode = (str) =&gt; btoa(unescape(encodeURIComponent(str)));
  const decode = (str) =&gt; {
    try { return str ? decodeURIComponent(escape(atob(str))) : ''; }
    catch { return str || ''; }
  };

  const encodedFiles = files.map(f =&gt; ({
      name: f.name,
      content: encode(f.content)
  }));

  const response = await fetch(`${LOCAL_BACKEND_URL}/execute`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      files: encodedFiles,
      main_file: mainFile,
      language_id: languageId,
      stdin: encode(stdin || '')
    }),
  });

  if (!response.ok) {
      throw new Error(`Execution failed to connect to backend:
${response.statusText}`);
  }

  const data = await response.json();
  return data;
};

export const isMockMode = () =&gt; false;
```

CompileX Source Code Archive

Generated on 13/4/2026, 6:45:21 pm

Total Lines: 3079 | Total Files: 23